

Deploying a Web Application on AWS Elastic Beanstalk

Submitted by: Minnusree

Date: June 17, 2026

1. Introduction

AWS Elastic Beanstalk is a Platform as a Service (PaaS) offered by Amazon Web Services. It allows developers to deploy and manage web applications without worrying about the underlying infrastructure. Elastic Beanstalk automatically handles capacity provisioning, load balancing, scaling, and application health monitoring.

2. Objective

The objective of this task was to deploy a simple Python web application on AWS using Elastic Beanstalk and verify that it runs successfully using the generated URL.

3. Deployment Steps

Step 1: Create AWS Free Tier Account

Created an AWS Free Tier account at aws.amazon.com. The free tier provides 12 months of free access to core AWS services including Elastic Beanstalk.

Step 2: Create IAM Roles

Two IAM roles were created for Elastic Beanstalk:

1. **aws-elasticbeanstalk-service-role** - Allows Elastic Beanstalk to manage AWS resources. Created using IAM > Roles > Create Role > AWS Service > Elastic Beanstalk - Customization.
2. **aws-elasticbeanstalk-ec2-role** - Allows EC2 instances to perform required operations. Created using IAM > Roles > Create Role > AWS Service > EC2. Attached policies: AWSElasticBeanstalkWebTier, AWSElasticBeanstalkWorkerTier, AWSElasticBeanstalkMulticontainerDocker.

Step 3: Create Elastic Beanstalk Environment

Navigated to AWS Elastic Beanstalk > Create Environment with the following settings:

3. **Environment tier:** Web server environment
4. **Application name:** MinnuSree
5. **Environment name:** MinnuSree-env
6. **Domain:** minnusree

7. **Platform:** Python 3.14 running on 64bit Amazon Linux 2023
8. **Application code:** Sample application

Step 4: Configure Service Access

In Step 2 of environment creation, selected the IAM roles created earlier: Service role set to aws-elasticbeanstalk-service-role and EC2 instance profile set to aws-elasticbeanstalk-ec2-role. EC2 key pair was left blank as SSH access was not required.

Step 5: Configure Instance Type

Set the environment type to Single instance (free tier eligible) and instance type to t3.micro. This avoids AutoScaling group costs and is suitable for development and testing purposes.

Step 6: Deploy and Verify

After reviewing all settings, clicked Create environment. The environment was successfully created and the application was deployed. The application was verified by opening the generated URL in a browser.

4. Verification

The deployed application was verified successfully by accessing the following URL:

minnusree.ap-south-1.elasticbeanstalk.com

The browser displayed the message: "Welcome to Your Elastic Beanstalk Application - Congratulations! Your Python application is now running on your own dedicated environment in the AWS Cloud."

5. Conclusion

The web application was successfully deployed on AWS Elastic Beanstalk using a Free Tier account. Elastic Beanstalk simplified the deployment process by automatically handling the infrastructure setup, allowing focus on the application itself. The application is now live and accessible via the generated AWS URL.